Ömer Gölcük 38101

EEE 58003 VLSI Systems Design I Lab Report

Introduction

This experiment focuses on the design and performance evaluation of four different adder architectures: Ripple Carry Adder (RCA), Carry Skip Adder (CSA), Kogge–Stone Adder (KSA), and Brent–Kung Adder (BKA). Each architecture was constructed at the gate level in Verilog HDL and tested for both 16-bit and 32-bit word lengths. Functional verification was carried out in Vivado through comprehensive testbenches to ensure correctness under various input combinations. To analyze speed performance, post-synthesis delay measurements were obtained using the Synopsys 32 nm Standard Cell Library, from which the most efficient adder architecture was determined.

Adder Architectures

Ripple Carry Adder (RCA)

The Ripple Carry Adder (RCA) is the most fundamental adder topology used for binary addition. It is composed of a chain of cascaded full adders, where each stage adds a corresponding pair of input bits along with the carry generated from the previous stage. The carry signal "ripples" through the chain from the least significant bit (LSB) to the most significant bit (MSB), as illustrated in Figure 1.

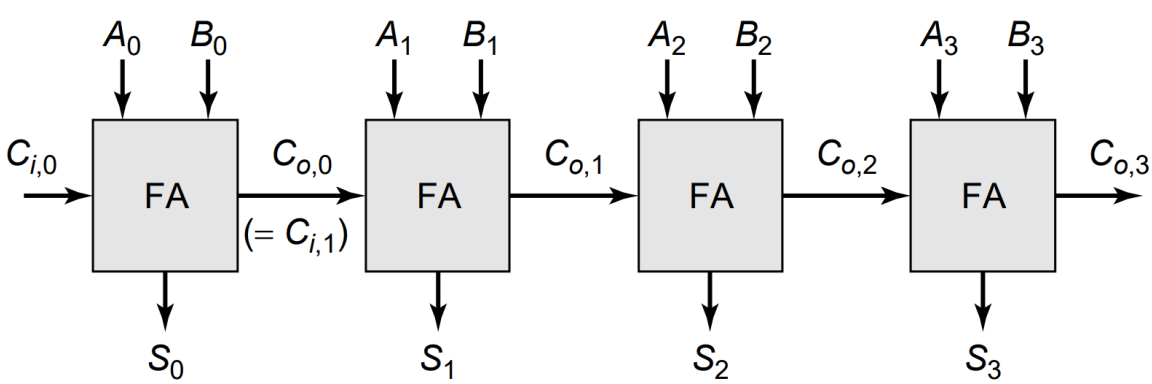


Figure 1: The Architecture for 4-bit Ripple Carry Adder

Because each full adder must wait for the carry output from the preceding stage, the RCA suffers from cumulative propagation delay. This means that as the bit-width increases, the total delay

grows linearly, resulting in a time complexity of *O(N)*. Despite this drawback, the RCA remains popular for its simplicity, compact design, and minimal hardware requirements.

The critical path of the RCA corresponds to the case where a carry generated in the least significant stage propagates through all subsequent stages before producing the final sum and carry outputs. Hence, the total delay of an *N*-bit RCA can be expressed as:

$$t_{delay} = (N-1) \cdot t_{carry} + t_{sum} \quad (1)$$

Here, t_{carry} denotes the delay associated with the propagation of carry signals, while t_{sum} represents the delay required for the final sum computation.

Carry Skip Adder – Carry Bypass Adder (CSA)

The Carry Skip Adder (CSA) is an improved adder architecture developed to mitigate the carry propagation delay inherent in the Ripple Carry Adder (RCA). In an RCA, each bit addition depends on the carry output from the previous stage, resulting in a delay that increases linearly with the bit-width. The CSA addresses this limitation by introducing a mechanism that allows the carry to "skip" over certain groups of bits when possible.

In a CSA, the entire adder is divided into multiple blocks, each consisting of several full adders. Within each block, the addition process is identical to that of an RCA. However, if all bits in a block generate a condition that allows the carry to propagate (i.e., all propagate signals are high), the carry input can bypass the block entirely and move directly to the next block. This operation is controlled by a bypass logic implemented using a multiplexer, which determines whether the carry should propagate through the block normally or be skipped. The block propagate signal is typically defined as $BP = P_0P_1P_2P_3$ for a 4-bit block, as shown in Figure 2.

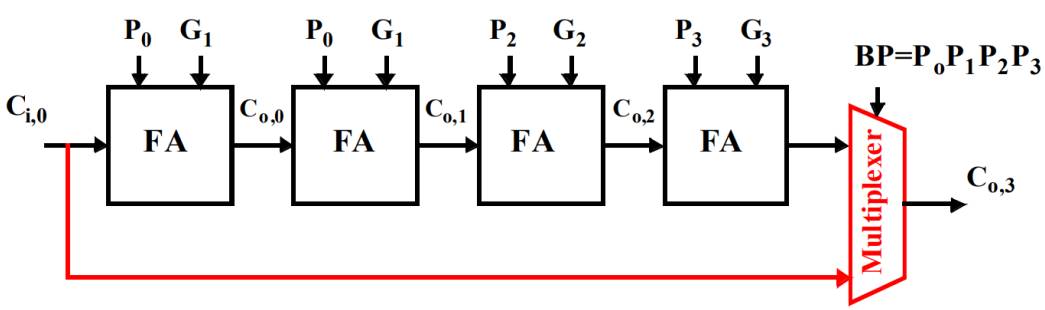


Figure 2: Architecture for 4-bit Carry Skip Adder

By allowing carry signals to skip over groups of adders, the CSA significantly reduces overall propagation delay, especially in wide adders. Although it introduces additional complexity and hardware due to the bypass logic, it achieves faster computation compared to a conventional RCA.

This improvement arises from minimizing the number of stages affected by sequential carry propagation.

The critical path delay of a Carry Skip Adder can be expressed as:

$$t_{delay} = t_{setup} + Mt_{carry} + (\frac{N}{M} - 1)t_{bypass} + (M-1)t_{carry} + t_{sum} \quad (2)$$

Here, t_{setup} represents the setup time, t_{carry} the delay for carry propagation within a block, t_{bypass} the delay through the multiplexer used for skipping, and t_{sum} the delay for generating the final sum output.

Kogge-Stone Adder (KSA)

The Kogge–Stone Adder (KSA) is a high-performance parallel prefix adder architecture specifically designed for fast arithmetic operations in wide-bit systems. Unlike linear adders such as the Ripple Carry Adder (RCA), the KSA employs a tree-like prefix structure that enables carry computation to occur in parallel rather than sequentially. This approach drastically reduces propagation delay, making it one of the fastest known adder topologies.

The operation of the KSA proceeds in multiple hierarchical stages. The first stage generates the *propagate* (P) and *generate* (G) signals for each bit position. Subsequent stages combine these signals through a network of prefix operations that iteratively merge and distribute carry information across all bit positions. This hierarchical process is illustrated in Figure 3, which shows the carry generation network for a 16-bit KSA.

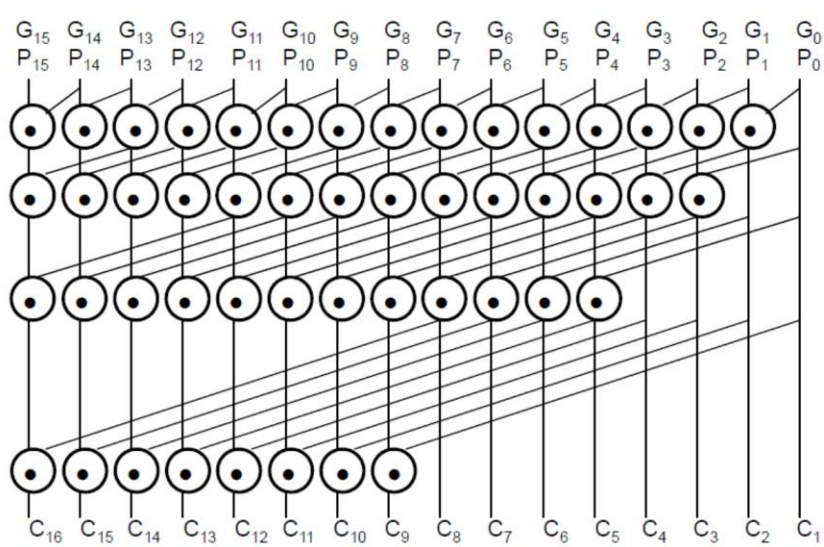


Figure 3: Architecture for 16-bit Kogge-Stone Adder

The prefix operations are executed using *dot operators* (·), mathematically defined as:

$$(G, P) \cdot (G', P') = (G + P \cdot G', P \cdot P') \qquad (3)$$

This formulation allows carry signals to be computed simultaneously for multiple bit positions without waiting for sequential propagation from lower-order bits. As a result, the KSA achieves logarithmic delay growth with respect to bit-width. The critical-path delay can be expressed as:

$$t_{delay} = t_{setup} + (\log_2 N)t_{dot} + t_{sum} \qquad (4)$$

The parallel prefix structure significantly accelerates carry propagation, providing superior speed compared to other adders. However, this advantage comes at the cost of increased circuit complexity and hardware usage due to the large number of logic gates and interconnections. Consequently, while the KSA consumes more area and power, its exceptional speed makes it ideal for high-performance and timing-critical applications such as processors and digital signal processors.

Brent Kung Adder (BKA)

The Brent–Kung Adder (BKA) is a parallel prefix adder architecture that achieves a balanced compromise between speed and hardware complexity. Similar to the Kogge–Stone Adder (KSA), it utilizes a hierarchical prefix structure to compute carry signals efficiently. However, the BKA is optimized to use fewer logic nodes and interconnections, making it more area- and power-efficient compared to the KSA. The BKA employs a logarithmic tree structure that first generates and propagates carry signals for each bit position and then combines them progressively using dot-operator logic. While both KSA and BKA share the same fundamental prefix principle, the BKA reduces hardware overhead by minimizing the number of prefix nodes and wiring density. This streamlined structure slightly increases delay compared to KSA but provides substantial savings in circuit area and power consumption. The overall carry-generation process for a 16-bit Brent–Kung Adder is illustrated in Figure 4.

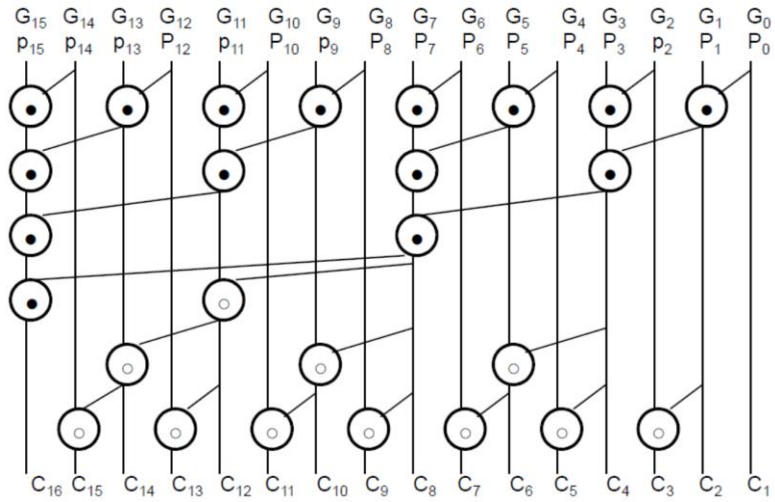


Figure 4: Architecture for Brent-Kung Adder

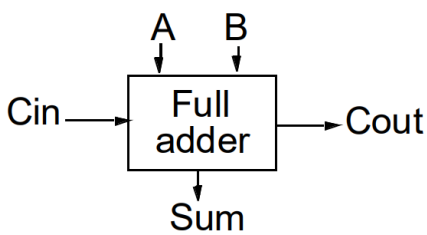
The critical-path delay of the BKA can be expressed as:

$$t_d = t_{setup} + (2\log_2 N - 2)t_{dot} + t_{sum} \qquad (5)$$

Although the BKA does not achieve the same propagation speed as the KSA, its reduced complexity leads to lower power consumption and smaller silicon area. This balance between speed and efficiency makes the Brent–Kung Adder a practical choice for applications where moderate performance is acceptable but hardware resources are constrained. Consequently, it is widely adopted in digital systems that demand an optimal trade-off between computational throughput and implementation cost.

Method

To implement the four adder architectures, the required logic gates—AND, OR, and XOR—were sourced from the provided Synopsys 32 nm standard cell library. The design process began with the construction of a full adder, which serves as the fundamental building block for the Ripple Carry Adder (RCA) and Carry Skip Adder (CSA). The truth table of the full adder, along with its corresponding generate, propagate, and delete states, is shown in Figure 5



A	B	C_{in}	AB (G)	$A \oplus B$ (P)	C_{out}	S	Carry Status
0	0	0	0	0	0	0	delete
0	0	1	0	0	0	1	delete
0	1	0	0	1	0	1	propagate
0	1	1	0	1	1	0	propagate
1	0	0	0	1	0	1	propagate
1	0	1	0	1	1	0	propagate
1	1	0	1	0	1	0	generate
1	1	1	1	0	1	1	generate

$$S = A \oplus B \oplus C_i$$

$$= A\bar{B}\bar{C}_i + \bar{A}B\bar{C}_i + \bar{A}\bar{B}C_i + ABC$$

$$C_o = AB + BC_i + AC_i$$

Figure 5: Truth Table of Full Adder

For the RCA, a basic 4-bit adder block was first designed using four cascaded full adders. By replicating and connecting these 4-bit modules, 16-bit and 32-bit RCA circuits were implemented. Specifically, the 16-bit RCA consists of four 4-bit blocks, while the 32-bit version contains eight such blocks. The CSA was then constructed by integrating these 4-bit RCA modules within each sub-block of the Carry Skip Adder structure. Each 4-bit RCA segment is connected to a 2-to-1 multiplexer that determines whether the carry should propagate through the block or bypass it entirely. This hierarchical composition of RCA blocks and multiplexers forms the complete Carry Skip Adder design. In contrast, the Kogge–Stone (KSA) and Brent–Kung (BKA) adders utilize a parallel prefix approach rather than a ripple-based one. Their implementation involves generating and combining *propagate* and *generate* signals through a sequence of dot-operator operations. Each stage in these adders' prefix trees systematically merges carry signals according to the equations defined in their respective architectures. This method drastically shortens carry propagation time compared to RCA and CSA designs. All adders were implemented and synthesized using Synopsys 32 nm Standard Cell Library to evaluate delay and performance characteristics.

Results

16-Bit Ripple Carry Adder

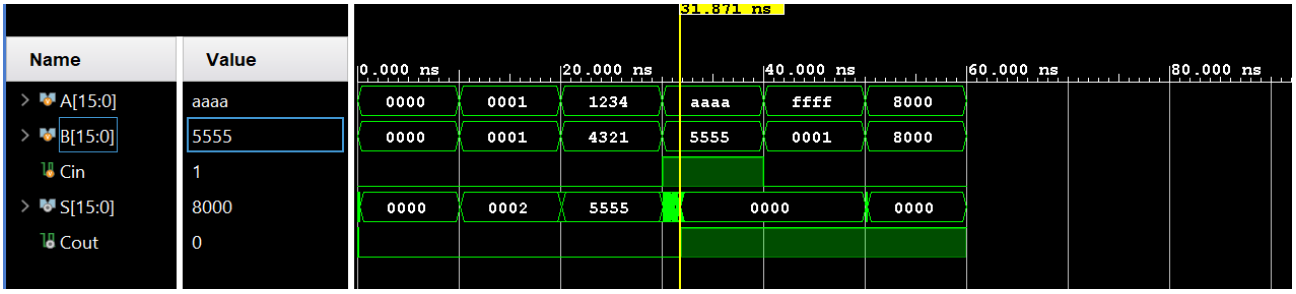


Figure 6: Addition Results with Several Cases for 16-Bit Ripple Carry Adder

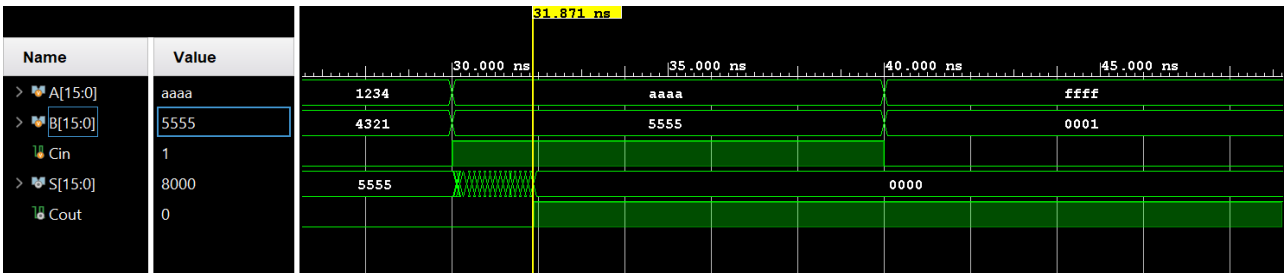


Figure 7:Critical Path Delay for 16-bit Ripple Carry Adder

32-Bit Ripple Carry Adder

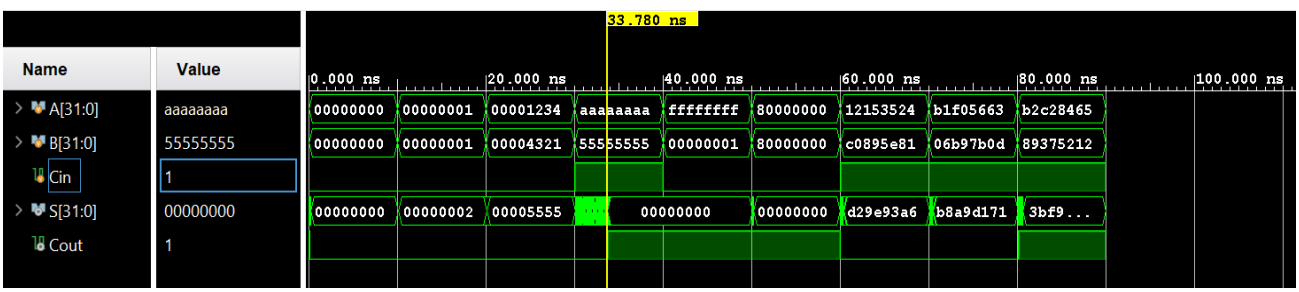


Figure 8: Addition Results with Several Cases 32-Bit Ripple Carry Adder

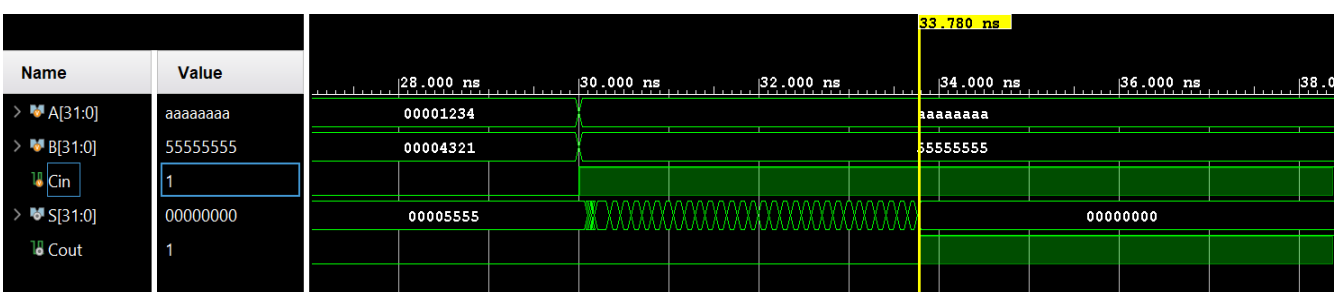


Figure 9:Critical Path Delay for 32-Bit Ripple Carry Adder

16-Bit Carry Skip Adder

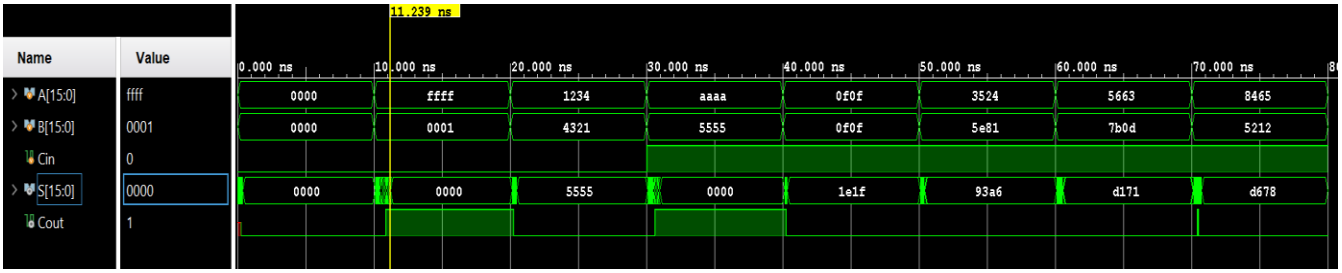


Figure 10: Addition Results with Several Cases for 16-Bit Carry Skip Adder

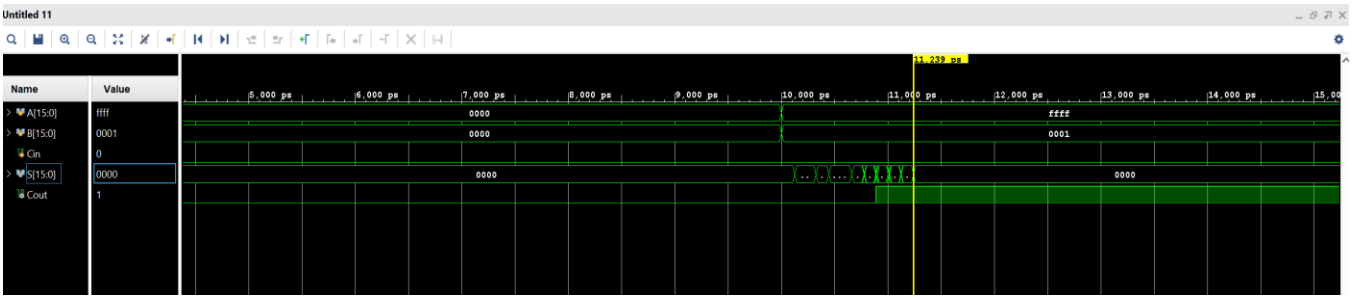


Figure 11: Critical Path Delay for 16-Bit Carry Skip Adder

32-Bit Carry Skip Adder

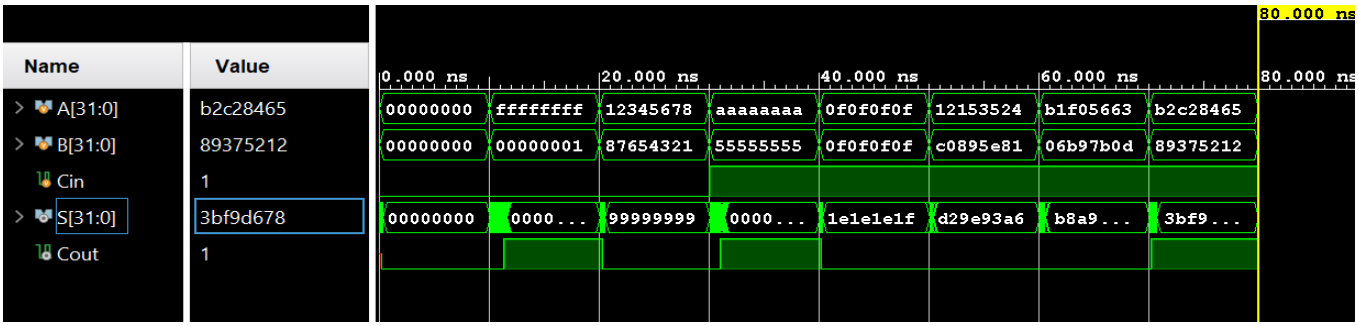


Figure 12: Addition Results with Several Cases for 32-Bit Carry Skip Adder

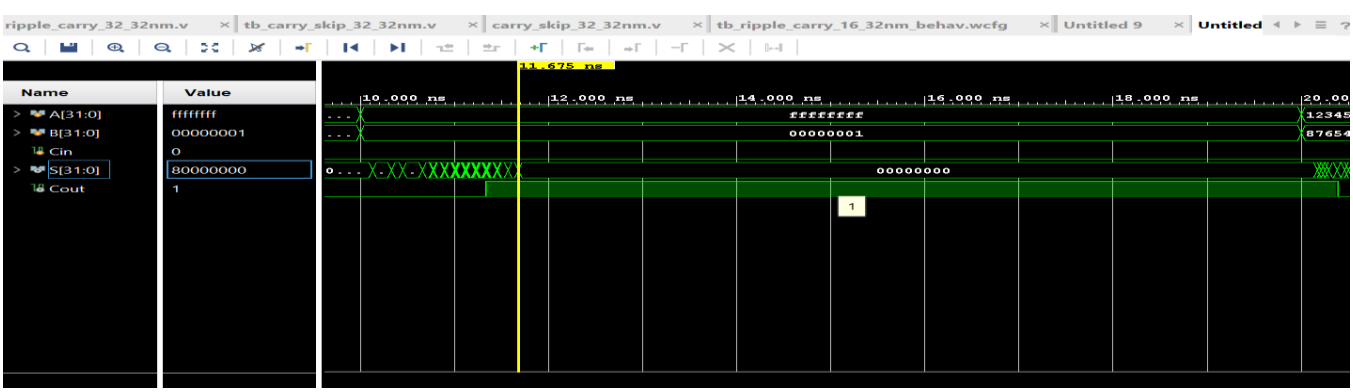


Figure 13: Critical Path Delay of 32-Bit Carry Skip Adder

16-Bit Kogge Stone Adder

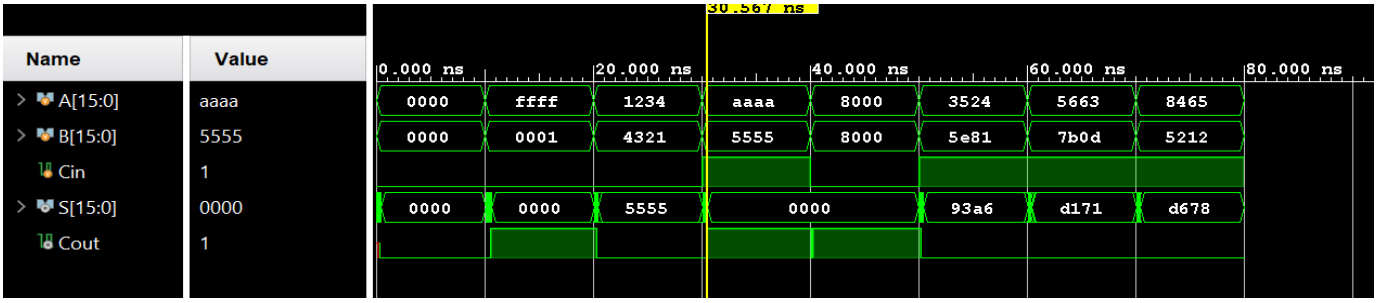


Figure 14:Addition Results with Several Cases for 16-Bit Kogge Stone Adder

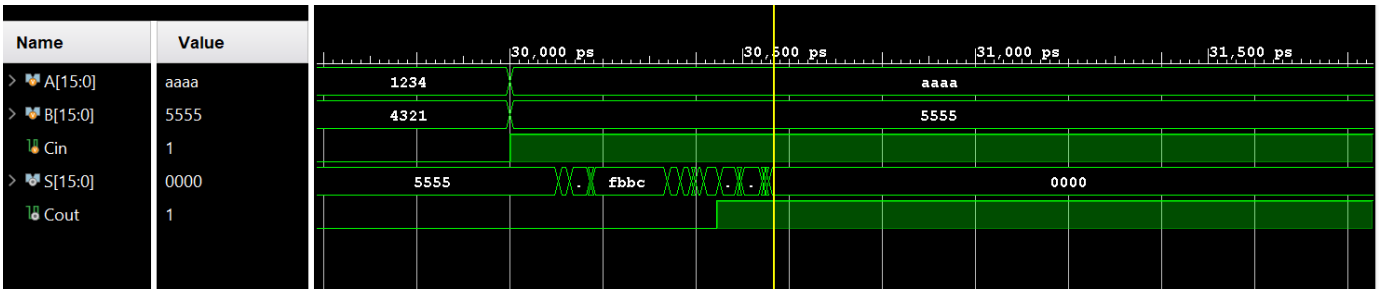


Figure 15:Critical Path Delay for 16-bit Kogge Stone Adder

32-Bit Kogge Stone Adder

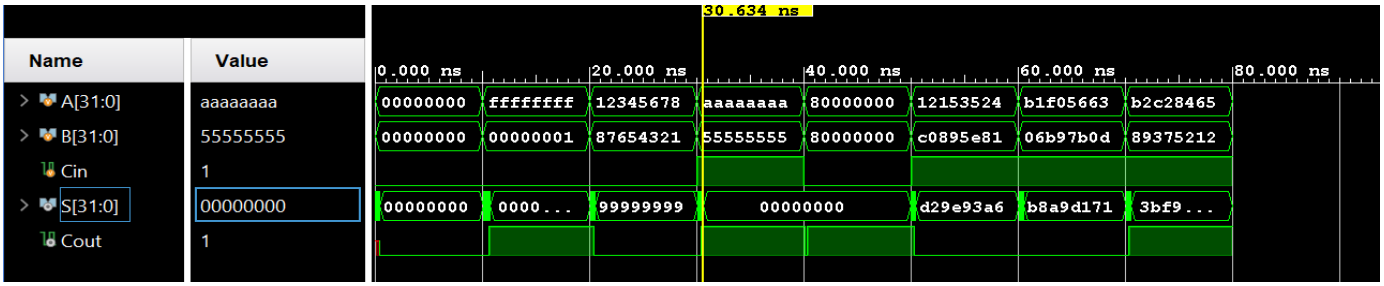


Figure 16:Addition Results with Several Cases for 32-Bit Kogge Stone Adder

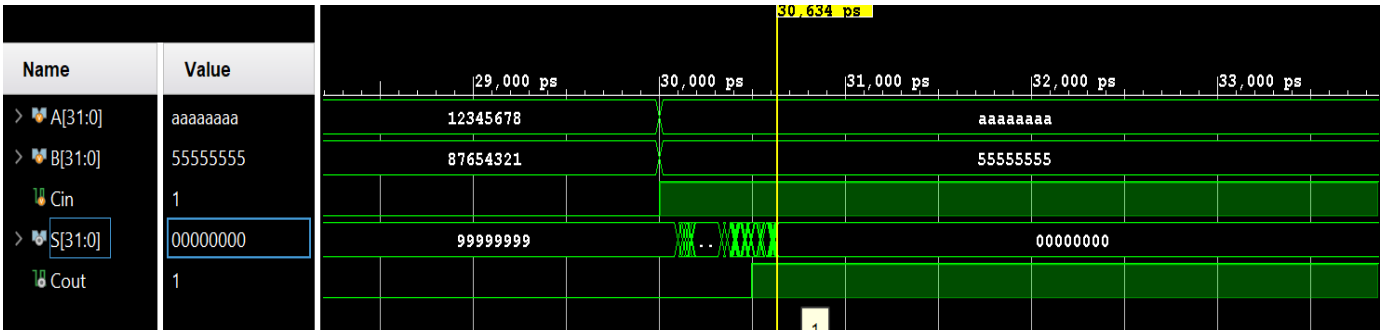


Figure 17: Critical Path Delay for Kogge Stone Adder

16-Bit Brent-Kung Adder

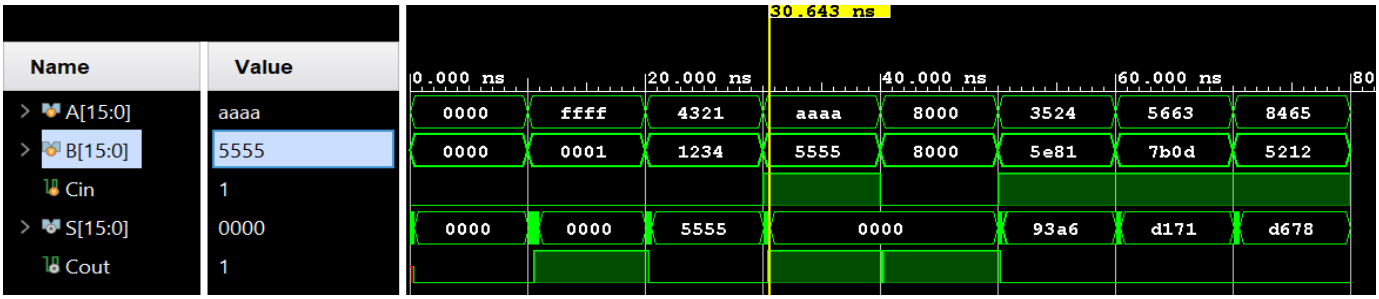


Figure 18: Addition Results with Several Cases for 16-Bit Brent-Kung Adder

Figure 19: Critical Path Delay for 16-Bit Brent Kung Adder

32- Bit Brent-Kung Adder

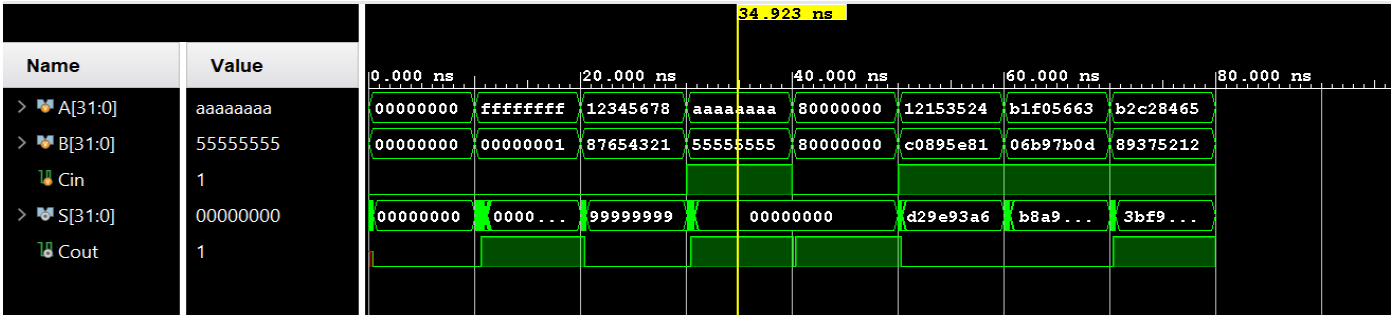


Figure 20:Addition Results with Several Cases for 32-Bit Brent Kung Adder

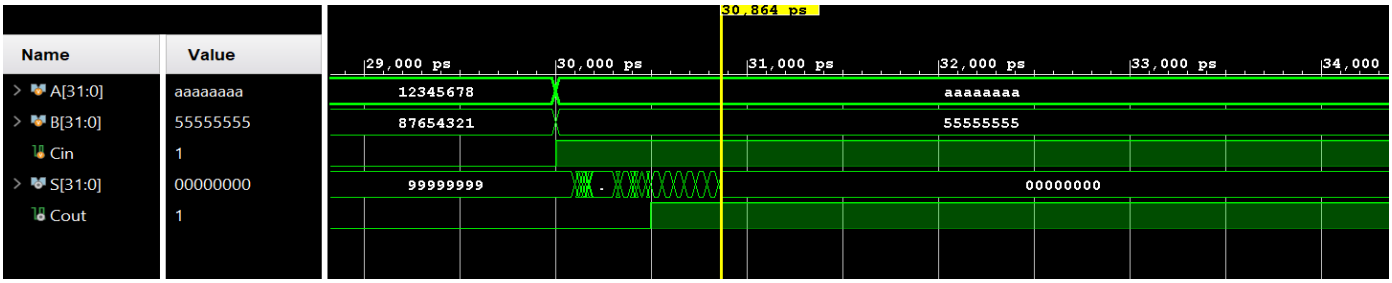


Figure 21:Critical Path Delay for 32-Bit Brent Kung Adder

Comparison of Adders

Each adder topology exhibits distinct trade-offs between speed, hardware complexity, and implementation cost. The Ripple Carry Adder (RCA) is the simplest in structure and requires the least hardware, but its carry must propagate sequentially through every bit, leading to the largest delay that scales linearly with bit-width. The Carry Skip Adder (CSA) improves upon this by introducing skip logic that allows the carry to bypass certain blocks when all propagate conditions are met, thereby reducing average delay while maintaining relatively low area overhead. The Kogge–Stone Adder (KSA), a parallel prefix architecture, achieves the fastest performance due to its highly parallel carry computation, exhibiting logarithmic delay growth with respect to bit-width; however, it incurs significant area, wiring, and power costs. The Brent–Kung Adder (BKA) offers a compromise between the two extremes—its tree structure provides faster operation than block adders like RCA and CSA while using fewer prefix nodes and interconnects than KSA, making it more efficient in terms of area and power. When the simulated critical-path delays obtained from Vivado are compared, these practical results align with the theoretical expectations: KSA shows the lowest delay, followed by BKA, CSA, and RCA in ascending order. This ranking illustrates how increasing levels of carry parallelism directly enhance addition speed at the expense of hardware complexity.

Adder Architecture	Delay for 16-bit (ps)	Delay for 32-bit (ps)
Ripple Carry	1871	3780
Carry Skip	1239	1675
Kogge Stone	567	634
Brent Kung	643	864

Table 1: The comparison of the different adder architectures

Discussion and Conclusion

After analyzing the delay results presented in Table I, several key observations can be made. First, the overall performance ranking of the designed adders, from fastest to slowest, is as follows: Kogge–Stone (KSA) → Brent–Kung (BKA) → Carry Skip (CSA) → Ripple Carry (RCA). This ordering is consistent with theoretical expectations. The parallel prefix architectures (KSA and BKA) outperform the multi-bit adders (CSA and RCA) because of their highly parallelized carry propagation networks. Between the two tree-based adders, KSA achieves lower delay due to fewer logic levels in its prefix tree. This trend is evident in both the 16-bit and 32-bit implementations. When comparing CSA and RCA, the skip logic in CSA reduces the effective carry chain length, resulting in better performance than RCA—again, in agreement with the simulation results.

In addition, the results show that delay scales with circuit complexity and bit-width. As the operand width increases from 16 bits to 32 bits, the delay grows proportionally depending on the adder

structure. For the RCA, the delay roughly doubles with bit-width since its carry propagation is linearly dependent on the number of bits (*O(N)*). The CSA exhibits slightly better scaling behavior due to the skip logic, achieving a sub-linear delay increase. In contrast, the parallel prefix adders (KSA and BKA) exhibit logarithmic delay growth (*O (log N)*) with respect to bit-width. For instance, when the bit-width doubles, the delay increases by roughly a factor of 1.25, aligning closely with the theoretical model. Overall, the simulation data are consistent with expected analytical behavior.

To conclude, this lab systematically investigated and compared the performance of four adder architectures—Ripple Carry Adder (RCA), Carry Skip Adder (CSA), Kogge–Stone Adder (KSA), and Brent–Kung Adder (BKA)—for both 16-bit and 32-bit implementations using Synopsys 32 nm library. Simulation results confirm that the parallel prefix adders (KSA and BKA) deliver the best performance due to their efficient tree-based carry computation. Among them, KSA proved the fastest owing to its minimal prefix depth, while BKA offered a balanced trade-off between speed, area, and power consumption. The CSA achieved moderate performance improvements over RCA thanks to its skip-logic optimization, whereas the RCA, being the simplest structure, exhibited the longest delay. In summary, this lab provides a clear understanding of how different adder topologies influence delay, complexity, and efficiency. It reinforces the theoretical principles of carry propagation and demonstrates the importance of using accurate standard cell models when evaluating high-speed digital designs.